OPENCLAW

+ Career-Ops Integration

AI Prompt — Design & Build Guide for SkillGraph v2

Auto-Pipeline URL → Eval + PDF + DB in one shot	**A–F Scoring Engine** 10 weighted dimensions, Indian market calibrated
Interview Story Bank Persistent STAR+R stories across all evaluations	**Negotiation Scripts** Per-offer salary leverage + counter-offer templates
Batch Sub-Agents 10+ jobs in parallel via Bull queue workers	**Portal Scanner** Naukri, Internshala, Greenhouse, Ashby, Lever, Unstop

Source project	github.com/santifer/career-ops — Career-Ops CLI pipeline
Target codebase	github.com/ftaakash/SkillGraph — SkillGraph v2 (Next.js 16+)
Integration target	app/(app)/openclaw/ — existing OpenClaw AI Job Agent module
Tech additions	Bull sub-agents, react-pdf, AmbitionBox scraper, STAR+R store, negotiate engine
Data scope	Indian job market: Naukri, Internshala, Wellfound, Unstop + global: Greenhouse, Ashby, Lever
Human-in-loop	AI evaluates + recommends. Student always makes the final apply decision.

THE MASTER AI PROMPT — Paste this into Claude / Cursor / Copilot

This is the single complete prompt to hand to any AI coding assistant. It contains everything needed to build the full Career-Ops feature set inside SkillGraph's existing OpenClaw module. Sections 1–7 below provide the detailed spec each part of this prompt refers to.

```
You are a senior full-stack engineer working on SkillGraph v2, a production-grade AI career
intelligence platform. The codebase is at github.com/ftaakash/SkillGraph.

Your task: Extend the existing OpenClaw AI Job Agent module (app/(app)/openclaw/) with the
complete feature set from github.com/santifer/career-ops. This is NOT a new project — you are
adding Career-Ops capabilities as sub-features of the existing OpenClaw page.

═══════════════════════════════════════════════════
EXISTING CODEBASE CONTEXT (READ FIRST)
═══════════════════════════════════════════════════

Framework: Next.js 16+ App Router strictly. Language: TypeScript (.tsx / .ts).
Styling: Tailwind CSS v4 via @theme in app/globals.css. No tailwind.config.ts.
UI: Radix UI + Shadcn components in components/ui/*
Animations: Framer Motion. All motion components require "use client".
Database: Prisma v5 with PostgreSQL. ORM at /prisma/schema.prisma.
Auth: NextAuth v5 beta with JWT sessions. Role in token: STUDENT | FACULTY | RECRUITER.
AI: OpenAI GPT-4o API via /app/api/ai/* route handlers.
Existing OpenClaw models in schema.prisma: OpenClawConfig, OpenClawListing,
OpenClawApplication.
Existing page: app/(app)/openclaw/page.tsx — already has AgentConfigPanel,
ApplicationFeed, MatchScoreCard components.

DO NOT:
- Create a new project or separate CLI. Integrate into existing files only.
- Add tailwind.config.ts. Use globals.css @theme for any new CSS variables.
- Use any UI library other than Shadcn + Radix.
- Change the existing dark industrial aesthetic (#0A0D14 bg, cyan/blue glows).
- Submit any job application without explicit student confirmation ("Apply" button click).

═══════════════════════════════════════════════════
FEATURE 1 — 10-DIMENSION A–F SCORING ENGINE
═══════════════════════════════════════════════════

Build: lib/openclaw/scorer.ts
Function: scoreJobListing(listing: OpenClawListing, student: User): Promise<EvaluationResult>

Score 10 weighted dimensions (total = 100 points). Assign letter grade:
A = 90+ B = 75-89 C = 60-74 D = 45-59 F = <45

Dimensions and weights:
D1 Role-Skill Match 25 pts — GPT-4o: student SkillProfile vs JD required skills
D2 Company Tier 15 pts — lookup IndiaMarketData.companyTier (FAANG=15, Unicorn=12, MNC=9, Service=5,
Startup=7)
```

D3 CTC vs Market Benchmark 15 pts — offer CTC vs IndiaMarketData.avgCtcLpa for role+city

D4 Growth Trajectory 10 pts — GPT-4o: analyse JD for seniority ladder signals

D5 Interview Complexity 10 pts — match JD interview style vs student sprint completion

D6 Location Preference 8 pts — city match vs OpenClawConfig.preferredCities

D7 Stack Excitement 7 pts — tech overlap between JD and student active learning gaps

D8 Application Timing 5 pts — fresh <7d=5pts, 7-14d=3pts, 15-30d=1pt, stale=0pt

D9 Culture Signal 3 pts — GPT-4o: AmbitionBox review sentiment for company

D10 Pipeline Integrity 2 pts — auto-0 if same company+role already in OpenClawApplication

Return type EvaluationResult:

```
{
totalScore: number,
grade: 'A' | 'B' | 'C' | 'D' | 'F',
dimensions: Record<string, { score: number, maxScore: number, reasoning: string }>,
recommendation: 'AutoApply' | 'ReviewAndApply' | 'Skip',
keyStrengths: string[],
keyWeaknesses: string[],
negotiationLeverage: string // 1 sentence: why student has / lacks leverage
}
```

Store result in new Prisma model: EvaluationResult (see Schema section below).

■■

FEATURE 2 — AUTO-PIPELINE (URL → FULL EVAL + PDF)

■■

Build: POST /api/openclaw/pipeline

Input: { jobUrl: string, studentId: string }

Pipeline:

1. Playwright scrapes the job URL — extract JD text, company, role, location, CTC if listed

2. Upsert into OpenClawListing (platform auto-detected from URL domain)

3. Call scoreJobListing() → EvaluationResult

4. If grade >= C and recommendation != 'Skip':

a. Call /api/resume/build with JD + student profile → tailored resume JSON

b. Generate PDF via react-pdf → upload to Cloudinary → create ResumeVersion record

5. Create OpenClawApplication with evaluationId linked

6. Return full pipeline result in one response

This is the "paste URL, get everything" feature. No separate steps for the student.

■■

FEATURE 3 — INTERVIEW STORY BANK

■■

Build: lib/openclaw/storybank.ts + Prisma model InterviewStory

After every evaluation with grade A or B, extract and upsert STAR+Reflection stories:

GPT-4o prompt: "Extract 2-3 STAR+Reflection behavioural interview stories from this student's resume and project history that would resonate for a {role} at {company}. Each story must cover:

Situation, Task, Action, Result, Reflection (what you'd do differently). Return JSON array."

Store each story in InterviewStory model with: storyId, userId, theme (Leadership/Conflict/Technical/Collaboration/Failure/Growth), applicableRoles[], companyContext, star_situation, star_task, star_action, star_result, star_reflection, usedForEvals[].

UI: Add "Story Bank" tab to /openclaw page showing all accumulated stories with filter by theme. Stories persist and improve across evaluations — the bank grows with every job evaluated.

■■■

FEATURE 4 — NEGOTIATION SCRIPTS

■■■

Build: lib/openclaw/negotiator.ts + UI component NegotiationScriptModal.tsx

Trigger: When application status moves to 'Offered' in OpenClawApplication.

GPT-4o generates 3 negotiation scripts per offer:

Script 1 — Geographic Discount Pushback: counter the "India rate" discount argument

Script 2 — Competing Offer Leverage: use other active applications as leverage

Script 3 — Skill Premium Frame: justify higher ask based on readiness score + skills

Each script contains:

openingLine, mainArgument, counterToExpectedObjection, closingAsk, talkingPoints[]

Store in new model: NegotiationScript linked to OpenClawApplication.

UI: Modal that appears when status = Offered, showing all 3 scripts with copy buttons.

Pre-fill with real student data:

- Student's readiness score and top skills
- Market CTC from IndiaMarketData for this role+city
- Any competing offers from other OpenClawApplication records with grade B+

■■■

FEATURE 5 — BATCH PROCESSING (SUB-AGENTS)

■■■

Build: lib/openclaw/batch-processor.ts

Extend existing Bull queue system (Upstash Redis).

New queue: 'openclaw-batch-eval'

Worker: processBatchEvaluation(job: { listingIds: string[], studentId: string })

- Spawn up to 5 concurrent sub-workers (Bull concurrency: 5)
- Each sub-worker: scoreJobListing() + generate PDF if grade >= C
- Aggregate results, rank by totalScore descending
- Send summary notification email via SendGrid on completion

UI addition on /openclaw page:

- "Batch Evaluate" button: select multiple listings from the feed → evaluate all in parallel
- Progress bar showing X/Total evaluated
- Results table sorted by grade, then totalScore

■■■

```
FEATURE 6 — PORTAL SCANNER (PRE-CONFIGURED)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Build: lib/openclaw/portal-scanner.ts

Extend existing Playwright scraper (lib/openclaw/scraper.ts).

Pre-configured portals (add to existing Naukri/LinkedIn/Internshala/Wellfound/Unstop scrapers):

Greenhouse: https://boards.greenhouse.io/{company}/jobs — parse job cards

Ashby: https://jobs.ashbyhq.com/{company} — parse JSON feed at /api/job-board/...

Lever: https://jobs.lever.co/{company} — parse job listing HTML

New UI element on /openclaw config panel:

"Portal Scan" section:

- Input: company name or direct Greenhouse/Ashby/Lever URL

- Scan button → scrapes all open roles → auto-evaluates each via scoreJobListing()

- Custom query: filter by role keyword before evaluating (e.g. "SDE", "ML")

Store all scraped listings in OpenClawListing as usual with platform field:

'Greenhouse' | 'Ashby' | 'Lever' | 'Naukri' | 'Internshala' | 'Wellfound' | 'Unstop' | 'LinkedIn'

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

FEATURE 7 — DASHBOARD TUI (PIPELINE VIEW)

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Extend: app/(app)/openclaw/page.tsx — add "Pipeline" tab alongside existing tabs.

Build component: components/openclaw/PipelineDashboard.tsx ("use client")

Design: Terminal-style table using the existing dark industrial aesthetic.

Background: #0A0D14, text: cyan/white, borders: dim cyan grid lines.

Columns: Company | Role | Grade | Score | CTC | Stage | Applied | Actions

Sorting: click column header to sort ascending/descending.

Filtering:

- Grade filter (A/B/C/D/F multi-select checkboxes)

- Stage filter (Applied/Screening/Interview/Offered/Rejected)

- Company tier filter

- Search by company name

Actions per row:

- "View Eval" → slide-over panel showing full 10-dimension breakdown

- "Story Bank" → show relevant stories for this company

- "Negotiate" → open NegotiationScriptModal (if stage = Offered)

- "Apply" → explicit confirmation dialog before any submission

Pipeline integrity:

- Highlight duplicate entries (same company + role) in amber

- Show "Applied X days ago" warning if re-applying to same company within 30 days

- Daily automated check: call POST /api/openclaw/integrity to merge dupes + normalise status

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

SCHEMA ADDITIONS (add to schema.prisma)
```

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

model EvaluationResult {
id String @id @default(cuid())
listingId String @unique
Listing OpenClawListing @relation(fields:[listingId], references:[id])
userId String
User User @relation(fields:[userId], references:[id])
totalScore Float
grade String // 'A'|'B'|'C'|'D'|'F'
dimensions Json // Record<string, {score, maxScore, reasoning}>
recommendation String
keyStrengths String[]
keyWeaknesses String[]
negotiationLeverage String
createdAt DateTime @default(now())
}

model InterviewStory {
id String @id @default(cuid())
userId String
User User @relation(fields:[userId], references:[id])
theme String // Leadership|Conflict|Technical|Collaboration|Failure|Growth
applicableRoles String[]
companyContext String
starSituation String @db.Text
starTask String @db.Text
starAction String @db.Text
starResult String @db.Text
starReflection String @db.Text
usedForEvals String[] // EvaluationResult IDs
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt
}

model NegotiationScript {
id String @id @default(cuid())
applicationId String @unique
Application OpenClawApplication @relation(fields:[applicationId], references:[id])
script1 Json // geographic discount pushback
script2 Json // competing offer leverage
script3 Json // skill premium frame
createdAt DateTime @default(now())
}

// Add to OpenClawApplication:
```

```
evaluationId String?
Evaluation EvaluationResult? @relation(...)
resumeVersionId String?
ResumeVersion ResumeVersion? @relation(...)

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
GPT-4o PROMPTS (exact strings to use)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

[SCORING — D1 Role-Skill Match]
System: "Score 0-25 how well this student's skills match this job. Return JSON:
{score: number, reasoning: string, matchingSkills: string[], missingCritical: string[]}.
Only the score and reasoning are required. Be calibrated for Indian tech hiring standards."
User: "Student skills: {skillsJSON}\nJD: {jdText}"

[SCORING — D4 Growth Trajectory]
System: "Score 0-10 the growth trajectory of this role for a fresh graduate.
Factors: seniority ladder clarity, mentorship signals, product vs service, equity mention.
Return JSON: {score: number, reasoning: string}"
User: "{jdText}"

[SCORING — D9 Culture Signal]
System: "Score 0-3 company culture based on AmbitionBox review sentiment.
Focus: work-life balance, manager quality, learning opportunity, politics.
Return JSON: {score: number, sentiment: positive|neutral|negative, keySignals: string[]}"
User: "Company: {company}\nAmbitionBox data: {ambitionboxText}"

[INTERVIEW STORY EXTRACTION]
System: "Extract 2-3 STAR+Reflection behavioural stories from this student profile that would
resonate for a {role} at {company}. Each story: situation, task, action, result, reflection
(what you'd do differently). Return JSON array of story objects."
User: "Resume text: {resumeText}\nTop projects: {projectsJSON}"

[NEGOTIATION SCRIPT — Script 1: Geographic Pushback]
System: "Write a salary negotiation script pushing back on geographic discount pricing.
Student data: readiness score {score}%, target role {role}, market rate {marketCTC} LPA.
Return JSON: {openingLine, mainArgument, counterToObjection, closingAsk, talkingPoints[]}"
User: "Offer details: {offerDetails}\nStudent competing offers: {otherOffers}"

[NEGOTIATION SCRIPT — Script 2: Competing Offer Leverage]
System: "Write a negotiation script using competing offers as leverage.
Keep it professional, not ultimatum-style. Indian corporate communication norms.
Return JSON: {openingLine, mainArgument, counterToObjection, closingAsk, talkingPoints[]}"
User: "Current offer: {offerDetails}\nCompeting offers: {competingOffers}"

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
BUILD ORDER (implement in this exact sequence)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
Step 1: Add 3 new Prisma models (EvaluationResult, InterviewStory, NegotiationScript) +
update OpenClawApplication with evaluationId + resumeVersionId.
Run: npx prisma migrate dev --name career_ops_integration

Step 2: Build lib/openclaw/scorer.ts — the 10-dimension scoring engine.
Test with mock listing and student data. Verify grade A–F assignment.

Step 3: Build POST /api/openclaw/pipeline — the auto-pipeline route handler.
Test with a real Naukri URL. Verify end-to-end: scrape → score → PDF → DB.

Step 4: Build lib/openclaw/storybank.ts — story extraction and upsert logic.
Add InterviewStory API route: GET/POST /api/openclaw/stories.

Step 5: Build lib/openclaw/negotiator.ts + NegotiationScriptModal.tsx component.
Wire to OpenClawApplication status change webhook.

Step 6: Extend lib/openclaw/batch-processor.ts with Bull concurrency 5 sub-agents.
Add batch evaluate UI to /openclaw page.

Step 7: Extend lib/openclaw/scraper.ts with Greenhouse, Ashby, Lever portal scrapers.
Add Portal Scan section to AgentConfigPanel.tsx.

Step 8: Build components/openclaw/PipelineDashboard.tsx — terminal-style pipeline table.
Add "Pipeline" tab to app/(app)/openclaw/page.tsx.

Step 9: Add POST /api/openclaw/integrity — dedup + status normalisation cron.
Schedule via Upstash QStash daily at 06:00 IST.

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
CRITICAL RULES (never violate)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

1. HUMAN-IN-LOOP: Never auto-submit a job application. The "Apply" button must always
require an explicit click with a confirmation dialog. AI recommends, student decides.

2. DARK AESTHETIC: All new UI must use the existing #0A0D14 background, cyan/blue glow
utilities, and Framer Motion PageTransition. No white backgrounds in authenticated views.

3. TAILWIND v4: Use @theme CSS variables in globals.css. Never add tailwind.config.ts.

4. SHADCN ONLY: All new components use Shadcn primitives (Button, Dialog, Table, Tabs,
Badge, Progress, Sheet). No additional UI libraries.

5. INDIA-FIRST DATA: All CTC figures in ■ LPA. All scoring calibrated for Indian hiring.
Mention Naukri/Unstop/Internshala before LinkedIn/Greenhouse in any platform list.

6. ADDITIVE SCHEMA: Never modify existing Prisma models — only add new fields/relations.
All new models are additions. Run prisma migrate dev, never db push in production.

7. TYPE SAFETY: Never use 'any'. All new functions fully typed. Resolve all TypeScript
errors including Shadcn/Recharts/react-day-picker version conflicts with generic bypasses
as documented in the existing ai_architecture_docs.md.

Begin with Step 1. Output the complete schema.prisma additions.
Wait for confirmation before proceeding to Step 2.
```

1. Career-Ops Features — Full Reference Table

All 10 features from Career-Ops (github.com/santifer/career-ops) mapped to their SkillGraph implementation. This matches the feature table from the Career-Ops README.

Feature	Description
Auto-Pipeline	Paste a job URL → full 10-dimension evaluation + ATS PDF generated + OpenClawApplication record created — one a
6-Block Evaluation	Structured A–F grade per job: Role–Skill Match, Company Tier, CTC vs Benchmark, Growth Trajectory, Interview Comp
Interview Story Bank	Accumulates STAR+Reflection stories across all evaluations. Maintains 5–10 master stories that can answer any behav
Negotiation Scripts	Per-offer salary negotiation framework: geographic discount pushback, competing offer leverage scripts, counter-offer te
ATS PDF Generation	Keyword-injected, per-JD CV generated via react-pdf — Space Grotesk + DM Sans typography. Stored as versioned Re
Portal Scanner	Pre-configured scrapers for Naukri, Internshala, Wellfound, Unstop, LinkedIn India + Greenhouse, Ashby, Lever via Play
Batch Processing	Evaluate 10+ job listings in parallel using sub-agent workers via Bull queues. Each worker independently scores, tailors
Dashboard TUI	Terminal-style UI within /openclaw page — browse, filter, sort pipeline by grade, company tier, CTC, stage. Built with the
Human-in-the-Loop	AI evaluates and recommends. Student decides. System never submits without explicit "Apply" confirmation. All auto-ap
Pipeline Integrity	Automated merge, dedup (same company+role → skip), status normalisation (Applied/Screening/Interview/Offered/Reje

2. A–F Scoring Engine — 10 Dimensions

The core intelligence of the Career-Ops integration. Every job listing is scored across 10 weighted dimensions calibrated for the Indian tech market. The total score maps to a letter grade (A=90+, B=75–89, C=60–74, D=45–59, F=below 45) which drives the recommendation: AutoApply, ReviewAndApply, or Skip.

ID	Dimension	Weight	Scoring Logic
D1	Role–Skill Match	25	How closely JD required skills map to student SkillProfile. Primary gate.
D2	Company Tier & Reputation	15	FAANG / Unicorn / MNC / Service / Startup via IndiaMarketData.companyTier.
D3	CTC vs Market Benchmark	15	Offer CTC vs. IndiaMarketData.avgCtcLpa for that role+city. Penalises lowballs.
D4	Growth Trajectory	10	Role seniority ladder, internal mobility signals scraped from AmbitionBox reviews.
D5	Interview Complexity	10	DSA-heavy vs System Design vs Balanced — weighed vs student's sprint readiness.
D6	Location & Logistics	8	City preference match, remote-first flag, relocation assistance mentioned in JD.
D7	Stack Excitement Score	7	Overlap between JD tech stack and technologies the student is actively learning.
D8	Application Timing	5	Days since posting. Fresh (<7d) = bonus. Stale (>30d) = penalty. Via scrapedAt.
D9	Company Culture Signal	3	Positive/negative keywords in AmbitionBox reviews (work-life, manager quality).
D10	Pipeline Integrity Check	2	Internal dedup check — same company+role not applied twice. Auto-skip if exists.

Grade → recommendation mapping:

A (90+) **AutoApply**
Outstanding match. Agent proceeds to resume tailoring + application immediately (with HITL confirm). Student gets push notification.

B (75–89)	**ReviewAndApply** Strong match. Application card surfaced at top of dashboard with "Apply Now" prominent CTA. Resume tailored and ready.
C (60–74)	**ReviewAndApply** Reasonable match. Shown in dashboard with yellow badge. Resume tailored only if student manually approves.
D (45–59)	**Skip** Weak match. Logged to pipeline as "Low Match" — visible but no resume generated. Student can override.
F (<45)	**Skip** Poor match or pipeline integrity violation (duplicate). Filtered from main feed. Accessible via "All" filter.

3. Auto-Pipeline — URL to Full Output

The "paste a URL, get everything" feature. Student pastes any job URL from any supported platform. The pipeline handles scraping, scoring, resume generation, PDF creation, and database logging in a single POST request — returning a complete result object.

What happens	What the student gets
• Input: any job URL from Naukri, Internshala, Wellfound, Unstop, LinkedIn India, Greenhouse, Ashby, or Lever	• Result JSON returned immediately to frontend for display
• Playwright auto-detects platform from URL domain and uses the correct scraper	• OpenClawApplication record created with evaluationId and resumeVersionId linked
• Full 10-dimension score computed immediately against live IndiaMarketData	• Student shown grade badge, key strengths/weaknesses, and recommendation
• ATS resume PDF auto-generated for any listing scoring grade C or above	• "Apply" button appears — explicit click required, never auto-submitted
• PDF uploaded to Cloudinary, linked as new ResumeVersion record	• Story bank scanned — relevant STAR stories surfaced for this company

API route — POST /api/openclaw/pipeline:

```
export async function POST(req: NextRequest) {
const session = await getServerSession(authOptions);
if (session.user.role !== 'STUDENT') return new Response('403', { status: 403 });

const { jobUrl } = await req.json();
const student = await prisma.user.findUnique({ where: { id: session.user.id },
include: { SkillProfile: true, OpenClawConfig: true } });

// Step 1: Scrape the job URL
const listing = await scrapeJobUrl(jobUrl); // auto-detects platform
await prisma.openClawListing.upsert({ ... });

// Step 2: Score the listing
```

```
const evalResult = await scoreJobListing(listing, student);
await prisma.evaluationResult.create({ data: { ...evalResult, listingId: listing.id } });

// Step 3: Generate resume if grade >= C (score >= 60)
let resumeVersion = null;
if (evalResult.totalScore >= 60) {
const resumeJson = await buildResumeForJd(student, listing.jdText);
const pdfUrl = await generateAndUploadPdf(resumeJson);
resumeVersion = await prisma.resumeVersion.create({
data: { userId: student.id, cloudinaryUrl: pdfUrl, targetJd: listing.jdText,
atsScore: resumeJson.atsScore, generatedBy: 'AI' }
});
}

// Step 4: Create application record (NOT auto-applied — awaits student confirm)
const application = await prisma.openClawApplication.create({
data: { userId: student.id, listingId: listing.id, matchScore: evalResult.totalScore,
resumeVersionId: resumeVersion?.id, status: 'Pending', evaluationId: evalResult.id }
});

// Step 5: Extract and upsert interview stories
if (evalResult.grade === 'A' || evalResult.grade === 'B')
await extractAndUpsertStories(student, listing, evalResult);

return Response.json({ listing, evaluation: evalResult, resumeVersion, application });
}
```

4. Interview Story Bank — Persistent STAR+R Engine

Every time a job is evaluated with grade A or B, GPT-4o extracts 2–3 STAR+Reflection behavioural stories from the student's resume and project history, contextualised for that specific company and role. Stories accumulate across all evaluations — by the time a student has evaluated 20 jobs, they have a rich, categorised bank of 5–10 master stories ready for any interview.

How stories are generated	How the student uses them
• Story themes: Leadership, Technical Challenge, Conflict Resolution, Collaboration, Failure/Learning, Growth	• Dashboard tab: "Story Bank" showing all stories filterable by theme
• Each story: Situation + Task + Action + Result + Reflection (what you'd do differently)	• Per-evaluation view: relevant stories for that specific company highlighted
• Stories tagged with applicableRoles[] and companyContext	• Pre-interview modal: shows top 3 most relevant stories for a given company+role
• Bank grows with every A/B evaluation — stories deduplicated and improved over time	• Stories linkable to actual evaluation records for context
• Student can manually add, edit, or promote stories in the Story Bank UI	• Export all stories as PDF for offline interview prep

```
// lib/openclaw/storybank.ts
const STORY_EXTRACT_PROMPT = `Extract 2-3 STAR+Reflection behavioural interview stories
from this student's profile that would resonate for a {role} position at {company}.
```

```
For each story provide:
- theme: one of Leadership|TechnicalChallenge|Conflict|Collaboration|Failure|Growth
- situation: context and challenge (2-3 sentences)
- task: what the student was responsible for (1-2 sentences)
- action: specific steps taken (3-4 sentences, focus on what THEY did)
- result: quantified outcome wherever possible (1-2 sentences)
- reflection: what they'd do differently and what they learned (1-2 sentences)
- applicableRoles: string[] of roles this story works for
- why_this_company: why this story is particularly relevant for {company}

Return ONLY a valid JSON array. No markdown, no explanation.`
```

5. Negotiation Scripts — Per-Offer Salary Leverage

When any application reaches status "Offered", GPT-4o generates three distinct negotiation scripts pre-filled with real student data: their readiness score, top skills, market CTC from IndiaMarketData, and any competing offers. Calibrated specifically for Indian corporate negotiation norms.

Three scripts generated	How they're surfaced + used
• Script 1 — Geographic Discount Pushback: counter the argument that India rates should be lower than global market	• Auto-triggered: modal appears when application.status → "Offered"
• Script 2 — Competing Offer Leverage: professional use of other active offers without ultimatums	• Pre-filled with real data: student readiness %, market CTC from IndiaMarketData
• Script 3 — Skill Premium Frame: justify higher ask based on readiness score + specific skill value	• Competing offers auto-pulled from other OpenClawApplication grade B+ records
	• Each script has: openingLine, mainArgument, counterToObjection, closingAsk
	• Copy button per script — paste directly into email or WhatsApp

6. Batch Processing + Portal Scanner

Batch Processing — evaluate 10+ listings in parallel:

Extends the existing Upstash Redis + Bull queue system with a new "openclaw-batch-eval" queue. Up to 5 concurrent sub-agent workers each independently score a listing, tailor a resume if grade ≥ C, and log the result. Completion triggers a SendGrid summary email.

UI behaviour	Queue internals
• Select multiple listings from ApplicationFeed → "Batch Evaluate" button	• Bull concurrency: 5 (5 workers in parallel)
• Progress bar: X of Total evaluated in real time (SSE or polling)	• Each worker: scoreJobListing() + optional resume generation
• Results table appears sorted by grade then totalScore	• Failed workers retry twice with exponential backoff
• SendGrid email on completion: summary table of all grades + recommended actions	• Total batch of 10 jobs completes in ~60-90 seconds

Portal Scanner — pre-configured company portal scraping:

Adds Greenhouse, Ashby, and Lever to the existing Playwright scraper alongside Naukri/Internshala/Wellfound/Unstop/LinkedIn India. Student pastes a company name or portal URL; the scanner scrapes all open roles and auto-evaluates each one via the scoring engine.

```
// lib/openclaw/portal-scanner.ts — platform detection + scrape

export async function scanPortal(input: string): Promise<OpenClawListing[]> {

// Auto-detect platform from URL or company name

if (input.includes('greenhouse.io'))

return scrapeGreenhouse(input); // boards.greenhouse.io/{co}/jobs

if (input.includes('ashbyhq.com'))

return scrapeAshby(input); // jobs.ashbyhq.com/{co} → /api/job-board/...

if (input.includes('lever.co'))

return scrapeLever(input); // jobs.lever.co/{co}

// Company name → try all three portals + existing Indian scrapers

const results = await Promise.allSettled([

scrapeGreenhouse(`https://boards.greenhouse.io/${slug(input)}/jobs`),

scrapeAshby(`https://jobs.ashbyhq.com/${slug(input)}`),

scrapeLever(`https://jobs.lever.co/${slug(input)}`),

scrapeNaukri(input), // existing

scrapeWellfound(input), // existing

]);

return results.flatMap(r => r.status === 'fulfilled' ? r.value : []);

}
```

7. New Prisma Schema Additions

Three new models added to the existing schema.prisma. Two existing models (OpenClawApplication, User) receive new optional fields. Run `npx prisma migrate dev --name career_ops_integration` — all changes are additive, nothing existing is broken.

Model / Field	Type	Purpose
EvaluationResult	id, listingId (unique), userId, totalScore (Float), grade (String), dimensions (Json), recommendation, keyStrengths[], keyWeaknesses	Full 10-dimension scoring result for one job listing. Stored once, retrieved on demand.
EvaluationResult.dimensions	Json — Record<D1-D10, {score, maxScore, reasoning}>	Per-dimension breakdown. Stored as JSON for flexible retrieval in the pipeline d
InterviewStory	id, userId, theme, applicableRoles[], companyContext, situation, task, action, result, reflection, timesUsed	Persistent STAR+R story bank. Grows across evaluations. Deduplicated by theme +
NegotiationScript	id, applicationId (unique), script1 (Json), script2 (Json), script3 (Json), competingOffer	Three negotiation scripts per offer. Each script: openingLine, mainArgument, co
OpenClawApplication (updated)	+ evaluationId (String?), + resumeVersionId (String?)	Two new optional FK fields linking to EvaluationResult and ResumeVersion. No
User (updated)	+ InterviewStory relation, + EvaluationResult relation	Two new hasMany relations. No column changes — pure relation additions.

8. File Structure — New Files Only

```
skillgraph-web/

■■■ lib/openclaw/

■ ■■■ scorer.ts NEW — 10-dimension scoring engine

■ ■■■ storybank.ts NEW — STAR+R story extraction + upsert
```

```
■ ■■■ negotiator.ts NEW — negotiation script generation
■ ■■■ batch-processor.ts EXTEND — add Bull concurrency + sub-agents
■ ■■■ portal-scanner.ts EXTEND — add Greenhouse, Ashby, Lever scrapers
■ ■■■ scraper.ts EXISTS — no changes needed (portal-scanner extends)
■ ■■■ matcher.ts EXISTS — no changes needed
■ ■■■ applicator.ts EXISTS — no changes needed
■
■■■ app/api/openclaw/
■ ■■■ pipeline/route.ts NEW — POST: URL → full eval + PDF + DB
■ ■■■ stories/route.ts NEW — GET all stories / POST upsert story
■ ■■■ negotiate/route.ts NEW — POST: generate 3 scripts for offer
■ ■■■ batch/route.ts NEW — POST: batch evaluate listing IDs
■ ■■■ integrity/route.ts NEW — POST: dedup + status normalise (cron)
■ ■■■ config/route.ts EXISTS — no changes
■ ■■■ agent/route.ts EXISTS — no changes
■ ■■■ listings/route.ts EXISTS — no changes
■ ■■■ applications/route.ts EXISTS — no changes
■
■■■ components/openclaw/
■ ■■■ PipelineDashboard.tsx NEW — terminal-style pipeline table
■ ■■■ EvaluationSlideOver.tsx NEW — full 10-dim breakdown slide panel
■ ■■■ NegotiationModal.tsx NEW — 3-script modal with copy buttons
■ ■■■ StoryBankPanel.tsx NEW — filterable story bank UI
■ ■■■ BatchEvalButton.tsx NEW — multi-select + progress bar
■ ■■■ PortalScanSection.tsx NEW — company input + scan UI
■ ■■■ AgentConfigPanel.tsx EXTEND — add PortalScanSection
■ ■■■ ApplicationFeed.tsx EXTEND — add grade badges + batch select
■ ■■■ MatchScoreCard.tsx EXTEND — add grade + recommendation display
■
■■■ app/(app)/openclaw/
■■■ page.tsx EXTEND — add Pipeline, Stories tabs
```

9. Build Order — Implement in This Exact Sequence

1 Schema migration — 3 new models + 2 field additions

npx prisma migrate dev --name career_ops_integration. Verify all new tables created. Check existing OpenClaw queries still work. No data loss.

2 lib/openclaw/scorer.ts — 10-dimension scoring engine

Build scoreJobListing(). Use IndiaMarketData for D2/D3. Test with 5 real Naukri listings. Verify grade A–F assignment is sensible.

3 POST /api/openclaw/pipeline — auto-pipeline route

Wire scraper → scorer → resume builder → PDF upload → DB. Test end-to-end with a real Naukri URL. Verify all 4 DB writes succeed.

4 lib/openclaw/storybank.ts + stories API route

Story extraction on grade A/B evaluations. Add Story Bank tab to /openclaw page with StoryBankPanel.tsx.

5 **lib/openclaw/negotiator.ts + NegotiationModal.tsx**

Trigger on status → Offered. Pre-fill with real student data. Test copy button for all 3 scripts.

6 **Batch processing extension — Bull concurrency 5**

Add openclaw-batch-eval queue. BatchEvalButton.tsx with progress bar. SendGrid summary on completion.

7 **Portal Scanner — Greenhouse + Ashby + Lever scrapers**

Extend scraper.ts. Add PortalScanSection.tsx to config panel. Test with a known Greenhouse URL (e.g. boards.greenhouse.io/anthropic/jobs).

8 **PipelineDashboard.tsx — terminal-style pipeline view**

Sort/filter by grade, stage, company. Per-row actions: View Eval, Story Bank, Negotiate, Apply. Pipeline integrity duplicate highlighting.

9 **POST /api/openclaw/integrity — daily dedup cron**

Schedule via Upstash QStash at 06:00 IST. Merge dupes, normalise status, flag stale applications (30+ days). Log results.